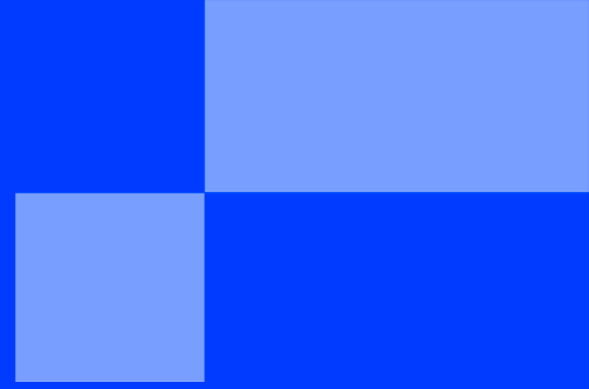
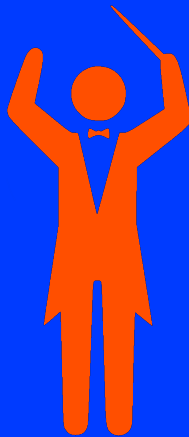




BERNSTEIN - BOOTSTRAP

< GRAB THE BATON AND BECOME AN
AMAZING CONDUCTOR />



BERNSTEIN - BOOTSTRAP

Using containers is an amazing way to ship code and to ease the deployment of applications and services, by removing the host infrastructure question burden.

However, up until now, you probably only worked with *single-host* deployments, meaning that all containers you had to run were run on the same machine.

While being an easy infrastructure to set up, it also comes with a set of cons, such as:

- ✓ If the host goes down, the entire application will be unavailable.
- ✓ If the load is increasing, the scalability is very limited.

Fortunately, a solution to manage the deployment of containers across a set of hosts (also known as a *cluster*) exists and is in great use in the DevOps world: it is the notion of *orchestration*.

Managing the *how*, *when*, *where* and *what* to run is part of an orchestrator duties.

During this bootstrap, you are going to become familiar with one of the most popular orchestration platform: [Kubernetes](#).



Step 0 – *Prelude to Kubernetes in YAML major, Op. 42*

A little quiz to start with

Before starting to use Kubernetes, it is important to gain some knowledge and to either acquire or clarify important points. Try to answer those questions with your friends:

- ✓ What is Kubernetes?
- ✓ What is a *cluster*?
- ✓ What is a *node*?
- ✓ What is a *pod*?
- ✓ In what way(s) are Kubernetes and Docker related?
- ✓ What is a Kubernetes *namespace*?
- ✓ What is the official Kubernetes 1.30 release name? (the answer will definitely surprise you)



Do not skip this part, you need to be able to answer the above questions to be at ease with using Kubernetes afterwards.

Setup time

Installing a full Kubernetes cluster can be difficult at first glance, and it will not be needed for this bootstrap.

We will use a single node **for this bootstrap only**, for the sake of simplicity, but keep in mind that **you will need a multi-node cluster for the project itself**.

Here is what will be needed for this bootstrap:

- ✓ *Docker*;
- ✓ *Minikube*, a simple tool to start a light Kubernetes cluster locally;
- ✓ *kubect**l*, a command-line tool for sending commands to Kubernetes.



You can make your life easier with `kubect``l` completion bash.

Your Mini Crewmate Minikube

Minikube automatically installs a Kubernetes cluster locally. Nice.

Try to figure out which 2-word commands allow you to do each of these actions, in order:

- ✓ *Start* Minikube.
- ✓ Get the *status* of Minikube.
- ✓ Get the *IP* address of Minikube.
- ✓ *Stop* Minikube.
- ✓ *Delete* the Minikube cluster.

Now that you know the five basics commands of Minikube, let us quickly configure `kubectl` (only needed the first time you are setting up Minikube):

```
Terminal
kubectl config current-context
kubectl config use-context minikube
```



Minikube must be running for these commands to work.

Do you `get` it?

`kubectl` allows you to `get` information on the different components of your cluster, including:

- ✓ nodes;
- ✓ namespaces;
- ✓ and pods.

Two-and-a-half quick questions:

- ✓ Which namespaces are automatically created on a fresh new cluster? What are their purpose?
- ✓ How to get pods from a specific namespace with `kubectl`?

Step 1 – Pod patrol

Create a simple pod named `hello-world` that runs a Docker container named `hello-world` with the following Docker image: `nginx`. Put it in a `hello-world.pod.yaml` file.

You can test your pod with the following command:

```
Terminal
kubect! apply -f hello-world.pod.yaml
```

Then wait a few seconds for the deployment.

Now that the pod is deployed, find the commands allowing you to perform each of the following actions:

- ✓ Listing existing pods in the cluster.
- ✓ Printing more details about the `hello-world` pod.
- ✓ Fetching and following the `hello-world` pod container's logs in real-time.
- ✓ Executing the `date` command **inside** the running `hello-world` pod container.
- ✓ Deleting the `hello-world` pod.

Step 2 – Environment variables for constant ecology

As always in DevOps, environment variables are the key to flexible, easy, and customizable deployments.

Update your `hello-world.pod.yaml` file and add the `FOO=bar` environment variable to the container.

Delete the pod and apply again the new configuration.

Now, check if it worked. You can either:

- ✓ execute `env` inside container;
- ✓ or describe the pod using `kubect! describe`.

Step 3 – Exposing ports, so that boats can dock(er)

The small application you are working with is a web server. Being able to access it would be nice, would it not?

Update your `hello-world.pod.yaml` file and ask Kubernetes to expose the port 80.

Re-apply your configuration by... deleting and applying `hello-world.pod.yaml` again. ;)

The port has been exposed, **but only inside the cluster**. To test your configuration, run:

```
Terminal
kubect1 port-forward [pod-name] 8080:80
curl localhost:8080/
```

`kubect1 port-forward` command builds a tunnel between your computer and the Kubernetes cluster (similar to `docker run -p` flag).

Now, can you:

- ✓ Print full configuration of the pod using `kubect1 describe`?
- ✓ See the exposed port?
- ✓ See the pod's IP address?

Step 4 – Internal DNS for external comfort

Thanks to the previous step, other services from within the cluster can request the HTTP server on `<pod-ip>:80`.

However, if the container dies and is recreated, its IP address will change. Not ideal.

But there is a solution: *Domain Name Systems* (a.k.a. *DNS*)!

They are easily set up in Kubernetes through what we call a *service*.

Create a Kubernetes service of type `ClusterIP` linked to the `hello-world` app, exposing port 80 to the same port on the cluster. Put its definition in a `hello-world.service.yaml` file.



To link the service to the app, you need to use what Kubernetes calls a *selector*. It is a **crucial** step in order to make the pod accessible through the service.

Apply the service's configuration (you already know the command ;)).

If you print the created services' information, you should see the pod's IP in the `Endpoints` field.

```
Terminal
kubect! describe service [service-name] | grep Endpoints
```

Now, you should be able to query your service from everywhere **within** the Kubernetes cluster:

```
Terminal
curl [service-name].[namespace].svc.cluster.local
```

You should then receive the HTML code of the NGINX welcome page.



Do not manually specify the cluster's IP address in your service, as it is subject to change.

Step 5 – Volumes (no joke here, because you do not joke about volumes)

Containers are ephemeral, their transient data are usually not important and are discarded when the container is deleted.

However, you sometimes need to persist data between container restarts (the biggest example being running a database).

Kubernetes provides a way to persist data between container restarts through *volumes*.

Most of the time, they consist of a pair of objects: a `PersistentVolume` and a `PersistentVolumeClaim`.



Before proceeding, take a few minutes to understand the difference between a `PersistentVolume` and a `PersistentVolumeClaim`, and how they complement each other. It is very important.

Create a 512 MB `PersistentVolume` and attach it via a `PersistentVolumeClaim` to the `hello-world` container.

Put both definitions in a `hello-world.volume.yaml` file, and apply them.

Step 6 – *Pods* have evolved into *deployments*!

Pods are nice, but *deployments* are the real boss stuff.

Deployments offer more features, such as:

- ✓ replication policies;
- ✓ deployment history (and rollback!);
- ✓ and auto-scaling rules.

The real question is: can you convert your pod into a deployment?

Prove that you can by putting its new definition in a `hello-world.deployment.yaml` file.

Finally, test it using all the knowledge you have acquired so far (you can do it!).

Step 8 – Wait, where is the step 7?

Thank you student, but the step 7 is in another document; namely, the project's document itself!

If you have reached this step and succeeded at the previous tasks, you are ready to grab the baton and to start conducting your very own symphony of containers.

Best of luck! :)

v 1.5

{EPITECH}